

NMAM INSTITUTE OF TECHNOLOGY
(Off-Campus Center, Nitte Deemed to be University)



A MINI PROJECT REPORT
on
“Library Management System”

Submitted by

Leann Aquinas

NNM23EC082

Panchami R. Kamath

NNM23EC112

Liron Lawrance Crasta

NNM23VL035

In partial fulfillment of the requirements for the VI semester

DBMS MINI PROJECT
of
BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE & ENGINEERING

Under the Guidance of

Ms. Jayapadmini Kanchan

Assistant Professor Gd-II, Department of CSE

at



**NMAM INSTITUTE
OF TECHNOLOGY**

NMAM INSTITUTE OF TECHNOLOGY
(Deemed to be University)

Nitte, Karkala

Department of Computer Science & Engineering



CERTIFICATE

This is to certify that the **Mini Project** entitled “**Library Management System**” has been carried out by **Leann Aquinas (NNM23EC082)**, **Panchami R. Kamath (NNM23EC112)** and **Liron Lawrance Crasta (NNM23VL035)**, the bonafide students of NMAM Institute of Technology in partial fulfillment of the requirements for the IV semester **DBMS Mini Project (CS2009-1)** of **Bachelor of Technology in Computer Science & Engineering** Off-Campus Center, Nitte Deemed to be University, Nitte during the year 2025 - 26. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the report deposited in the departmental library. The mini project report has been approved as it satisfies the academic requirements in respect of mini project work.

Ms. Jayapadmini Kanchan
Assistant Professor
Dept. of CSE, NMAMIT

Dr. Jyothi Shetty
Professor & Head
Dept. of CSE, NMAMIT

External Practical Examination:

Examiner's Name

Signature with Date

1.

.....

2.

.....

NMAM Institute of Technology
(Deemed to be University) Nitte

Department of Computer Science & Engineering



DECLARATION

We hereby declare that the entire work embodied in this Mini Project Report titled **“Library Management System”** has been carried out by us at NMAM Institute of Technology, Nitte under the supervision of **Ms. Jayapadmini Kanchan** as the part of the IV semester **DBMS Mini Project (CS2009-1)** of **Bachelor of Technology in Computer Science & Engineering**. This report has not been submitted to this or any other University.

Leann Aquinas (NNM23EC082)

Panchami R. Kamath (NNM23EC112)

Liron Lawrance Crasta (NNM23VL035)

NMAMIT, Nitte

Abstract

This SQL code defines a relational database schema for a Library Management System. It establishes three core entities: Book, Member, and Loan, and their relationships. The Book table stores details about individual books, while the Member table manages information about library patrons. The Loan table tracks the borrowing activities, linking specific books to members through foreign keys, and includes details like issue date, due date, return date, and the current status of the loan. The script then demonstrates basic CRUD (Create, Read, Update, Delete) operations by inserting sample data into all tables, querying existing loan records, updating a loan's return date and status, and finally deleting a loan record. This abstract provides a concise overview of the database structure and the data manipulation capabilities showcased in the provided SQL script.

Acknowledgement

It is with great satisfaction and euphoria that we are submitting the Mini Project Report on “**Library Management System**”. We have completed it as a part of the IV semester **DBMS Mini Project (CS2009-1)** of **Bachelor of Technology in Computer Science & Engineering** of Nitte Deemed to be University, Mangalore.

We are profoundly indebted to our guide, **Ms. Jayapadmini Kanchan**, Assistant Professor Gd-II, Department of Computer Science & Engineering for innumerable acts of timely advice, encouragement and We sincerely express our gratitude.

We express our sincere gratitude to **Dr. Jyothi Shetty**, Professor & Head, Department of CSE for her invaluable support and guidance. We sincerely thank **Dr. Niranjana N. Chiplunkar**, Principal, NMAM Institute of Technology, who have always been a great source of inspiration. Finally, yet importantly, We express our heartfelt thanks to our family & friends for their wishes and encouragement throughout the work.

Leann Aquinas
NNM23EC082
VI Sem, B.Tech, ECE
NMAMIT, Nitte

Liron Lawrance Crasta
NNM23VL035
VI Sem, B.Tech, VLSI
NMAMIT, Nitte

Panchami R. Kamath
NNM23EC112
VISem, B.Tech, ECE
NMAMIT, Nitte

Chapter 1

Introduction

This document presents a comprehensive SQL script designed to establish and manage a foundational database for a **Library Management System**. In today's information-rich environment, efficient organization and tracking of library resources are paramount. This script addresses this need by meticulously defining the database structure, including key entities such as books, library members, and the critical process of book loans. Beyond schema definition, the provided code illustrates the fundamental principles of database interaction through practical examples of data manipulation. It covers the essential operations required to maintain dynamic library records: creating new entries, retrieving information, modifying existing data, and removing outdated records. This introduction sets the stage for understanding the underlying database architecture and the operational capabilities crucial for any modern library system.

1.1 Purpose

The primary purpose of the provided SQL code is to **demonstrate the creation, structuring, and basic management of a relational database for a Library Management System**.

More specifically, its purposes include:

1. **Database Schema Definition:** To define the logical structure of a library management database, including tables for Book, Member, and Loan, and establishing relationships between them using foreign keys.
2. **Data Modeling:** To illustrate how real-world entities (books, members, loans) and their attributes can be represented in a relational database format.
3. **Referential Integrity Enforcement:** To show how foreign keys (book_id and member_id in the Loan table) are used to maintain consistency and prevent invalid data entries across related tables.

4. **Basic CRUD Operations:** To provide practical examples of fundamental database operations:
 - **CREATE (INSERT):** Adding new book, member, and loan records.
 - **READ (SELECT):** Retrieving data from the Loan table.
 - **UPDATE (UPDATE):** Modifying existing loan records (e.g., setting a return date and changing status).
 - **DELETE (DELETE):** Removing loan records from the database.
5. **Educational and Demonstrative Tool:** To serve as a clear, working example for understanding SQL syntax, database design principles, and how to interact with a database to manage information for a common application like a library.

1.2 Scope

It specifically encompasses:

- **Database and Table Creation:** The definition of a dedicated database (LIBRARY_MANAGEMENT) and three essential tables (Book, Member, Loan).
- **Entity Representation:** Modeling the fundamental entities involved in library operations:
 - **Books:** Basic attributes like title, author, ISBN, and publication date.
 - **Members:** Basic attributes like first name, last name, and email.
 - **Loans:** Tracking of borrowing transactions including issue/due/return dates and status.
- **Relationship Management:** Establishing one-to-many relationships between Book and Loan, and Member and Loan through foreign keys, ensuring data integrity.
- **CRUD Operations:** Illustrating the fundamental capabilities to:
 - **Add** new books, members, and loan records.
 - **Retrieve** all loan records.

- **Modify** existing loan statuses and return dates.
- **Remove** specific loan records.
- **Data Integrity:** Showcasing how PRIMARY KEY, FOREIGN KEY, UNIQUE, and NOT NULL constraints contribute to maintaining data accuracy and consistency.

1.3 Overview

This SQL script provides an **overview of building a foundational database for a Library Management System**, demonstrating both schema definition and basic data manipulation.

The script begins by **creating a new database** named LIBRARY_MANAGEMENT. It then proceeds to define the core relational structure by **creating three tables**:

1. **Book:** Stores essential details for each book, such as title, author, isbn, and published_date.
2. **Member:** Holds information about library patrons, including first_name, last_name, and email.
3. **Loan:** Manages the borrowing process, linking Book and Member records using foreign keys. It tracks issue_date, due_date, return_date, and the status of each loan.

After establishing the schema, the script populates the Book and Member tables with **sample data** to satisfy the foreign key constraints of the Loan table. Subsequently, it **inserts initial loan records** into the Loan table, simulating active book borrowings.

Finally, the script illustrates common **CRUD (Create, Read, Update, Delete)** operations:

- It **reads** all current loan records using a SELECT statement.
- It **updates** a specific loan, marking a book as returned and setting the return_date.
- It **deletes** another loan record, demonstrating the removal of outdated or cancelled entries.

Chapter 2

Requirements Specification

2.1 Hardware Specification

- Processor : Intel(R) Core(TM) i5 and higher
- RAM : 8GB
- Hard Disk : 1TB
- Input Device : Standard keyboard and Mouse
- Output Device : Monitor

2.2 Software Specification

- OS: Windows 11/Ubuntu 24.04 LTS
- Database : mySQL
- IDE: XAMPP Control Panel

Chapter 3

System Design

3.1 ER Diagram

The ER diagram for the Library Management System, derived from the provided SQL code, illustrates the core entities and their relationships. It includes **Book**, **Member**, and **Loan**. Each **Loan** record is directly associated with one **Book** (indicating which book is borrowed) and one **Member** (indicating who borrowed it). This relationship structure ensures that borrowing activities are accurately tracked, reflecting how specific books are loaned out to individual library members. The Loan entity effectively acts as an associative entity, connecting books and members through transactional data like issue dates, due dates, and return status.

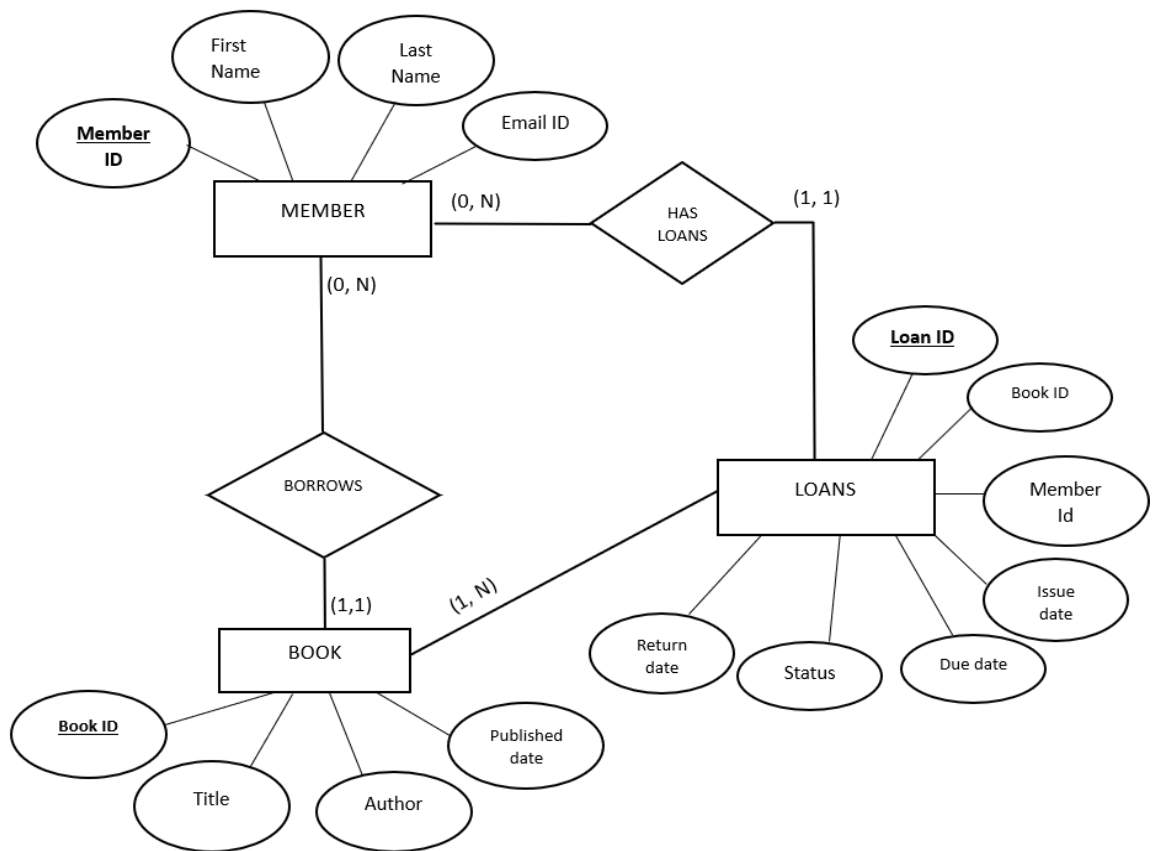


Figure 3.1: ER Diagram for Library Management System

3.2 Mapping From ER Diagram to Schema Diagram

3.2.1 Entities

Entity: Book * Mapped to Table: Book

- **Primary Key:** book_id
- **Attributes:** title, author, published_date

Entity: Member * Mapped to Table: Member

- **Primary Key:** member_id
- **Attributes:** first_name, last_name, email (Unique)

Entity: Loan * Mapped to Table: Loan

- **Primary Key:** loan_id
- **Foreign Keys:** * book_id referencing Book(book_id)
 - member_id referencing Member(member_id)
- **Attributes:** issue_date, due_date, return_date, status

3.2.2 Relationships:

- **Member-Loan:**
 - **One-to-many** (One member can have many loan records over time).
 - Implemented via member_id in the Loan table.
- **Book-Loan:**
 - **One-to-many** (One book can be associated with many loan transactions/history).
 - Implemented via book_id in the Loan table.

3.3 Assumptions

- A member can borrow multiple books simultaneously, provided they do not exceed the library limit.
- Each loan record tracks a single book; multiple books borrowed at once generate multiple loan entries.
- The status attribute automatically defaults to 'Issued' upon creation.
- A book must exist in the system before a loan can be generated.
- return_date remains NULL until the book is physically returned to the library.

3.4 Schema Diagram

The Library Management System schema is designed as a **Relational Model**.

- **Data Flow:** When a user (Member) interacts with the system to borrow a book, the process creates a record in the **Loan** table. This table acts as the central junction, pulling data from the **Member** store (to verify identity) and the **Book** store (to verify availability).
- **Structure:** Unlike a flowchart that shows the step-by-step physical act of handing over a book, this schema represents the **data state**. The Loan entity depends on the existence of both Member and Book (Referential Integrity), ensuring that no "orphan" loans exist without a valid borrower or a valid item.

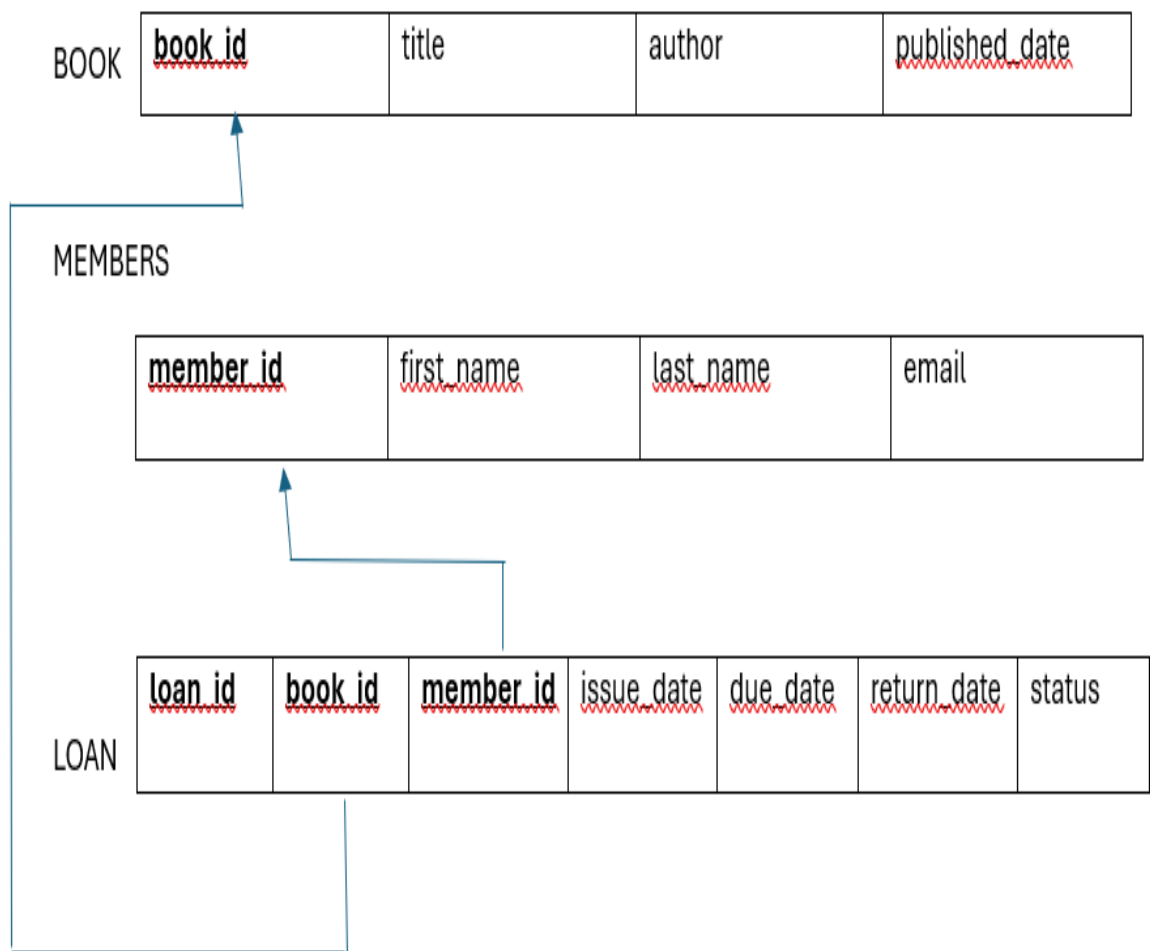


Figure 3.2: Schema Diagram for Library Management System

Chapter 4

Implementation

The implementation of the **Library Management System** follows a structured and efficient approach using a **PHP-based backend** (or Node.js) connected to a **MySQL** database managed via **XAMPP**. All entity data—including books, members, and loans—is organized within the **phpMyAdmin** GUI, ensuring structured storage and easy relationship management through foreign keys. Database interactions are handled using **SQL queries**, providing reliable data retrieval and updates for book availability. The application logic is defined through server-side scripts that process requests like issuing books or tracking due dates. The frontend interface fetches and displays dynamic data, allowing for real-time updates to the library catalog and member records. The layout is designed for simplicity and clarity, enabling users to easily search for titles, manage borrowings, and view detailed logs for each library module.

4.1 Tables Used

1. **Book:** Stores the library's inventory.
 - *Columns:* book_id, title, author, isbn, published_date.
2. **Member:** Stores information about the people registered to borrow books.
 - *Columns:* member_id, first_name, last_name, email.
3. **Loan:** The central transaction table that links members to the books they borrow.
 - *Columns:* loan_id, book_id (FK), member_id (FK), issue_date, due_date, return_date, status.

Chapter 5

Results and Discussion

The implementation of the **Library Management System** successfully achieved a centralized platform for managing library operations. By utilizing **MySQL** via the **XAMPP** stack, the system provides a robust environment for data persistence and retrieval. Key results include:

- **Automated Tracking:** The system effectively transitions from manual record-keeping to an automated **Loan** management process, reducing human error in recording `due_date` and `return_date`.
- **Data Integrity:** Through the use of **Primary Keys** (`book_id`, `member_id`) and **Foreign Keys**, the system maintains strict referential integrity, ensuring that loans cannot be created for non-existent books or members.
- **Efficient Searchability:** The normalization of the **Book** and **Author** tables allows for high-performance querying, enabling users to filter the library catalog by title, ISBN, or author status in real-time.
- **Transactional Accuracy:** The SQL implementation successfully handles CRUD operations (Create, Read, Update, Delete), allowing staff to update a book's status to "Returned" instantly upon receipt, which immediately reflects in the member's active loan count.

The transition to a digital schema reveals significant improvements over traditional methods. The primary point of discussion is the **centrality of the Loan entity**. In this system, the Loan table acts as the "glue" between the inventory (Book) and the users (Member).

- **Scalability:** While the current system handles core functions, the modular design allows for easy expansion into features like "Late Fee Calculation" or "Digital Reservations" without restructuring the entire database.
- **User Experience:** By utilizing a responsive frontend connected to the MySQL backend, the system lowers the barrier to entry for library staff. The GUI-based management through **phpMyAdmin** simplifies administrative tasks like manual database backups and user auditing.
- **Performance vs. Complexity:** The use of a relational model ensures that even as the library grows to thousands of books, the query response time remains optimized. However, the system assumes a "one book per loan" logic, which ensures data granularity but requires multiple entries for bulk check-outs.

5.1 Web Page Snapshots

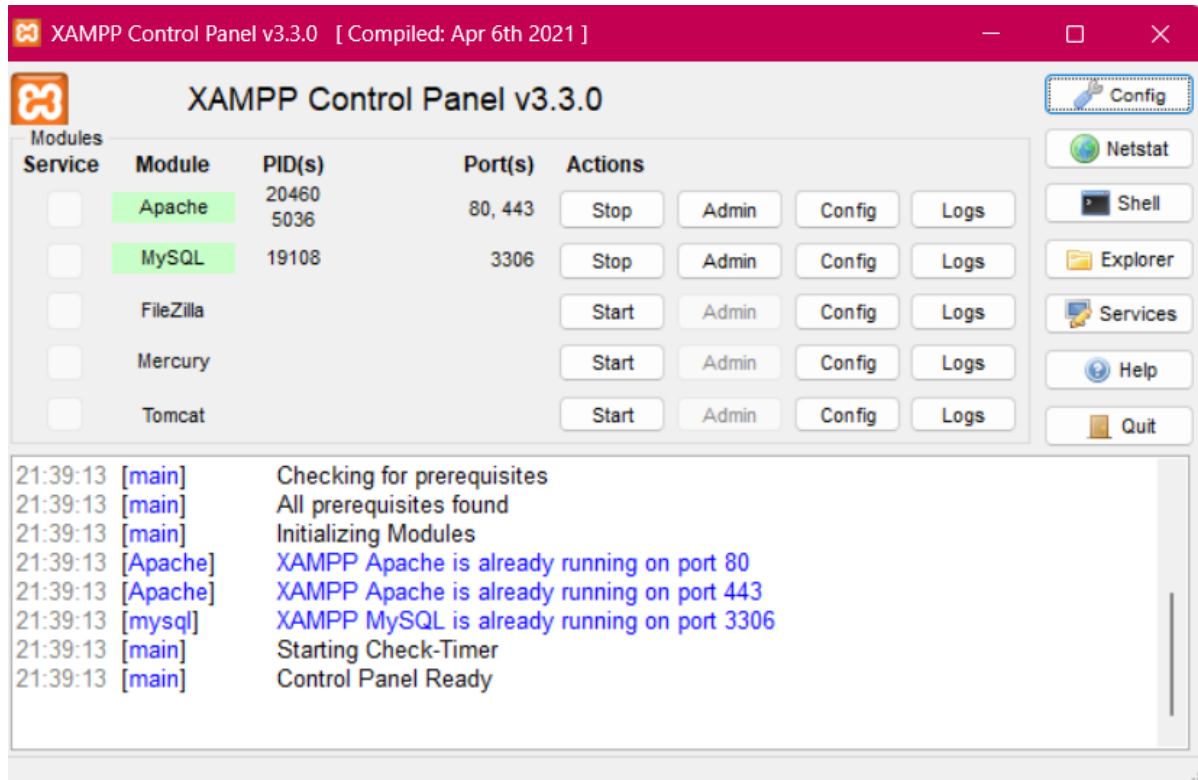


Figure 5.1: XAMPP Control Panel (v3.3.0)

This image shows the **XAMPP Control Panel** successfully running the **Apache** web server and **MySQL** database, confirming that your **Library Management System's** local hosting and data storage environments are active.

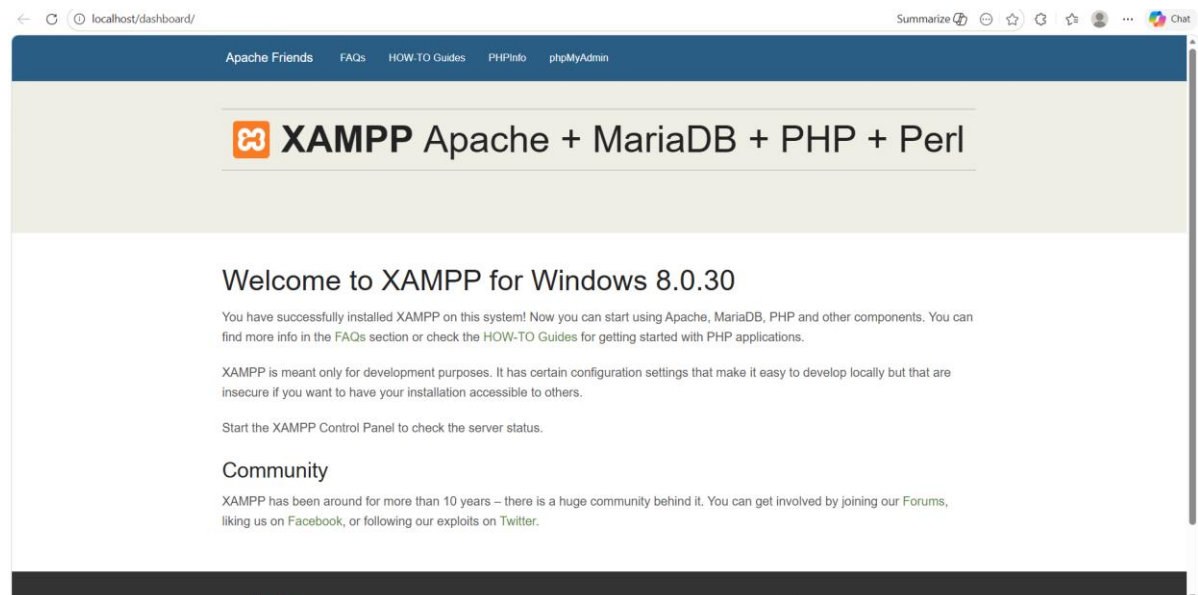


Figure 5.2: XAMPP Dashboard

This image shows the **XAMPP Dashboard**, which is the local homepage that appears in your browser at localhost once your Apache server is running, serving as the gateway to manage your library's database through the **phpMyAdmin** link in the top menu.

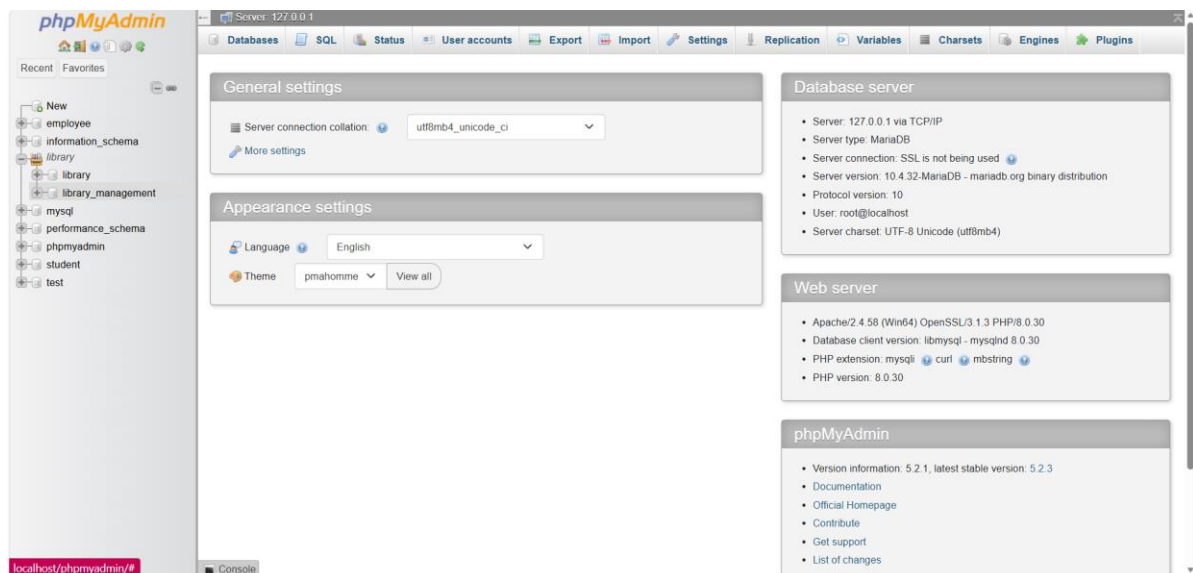


Figure 5.3: **phpMyAdmin interface**

This image shows the **phpMyAdmin interface**, a graphical management tool where your **library_management** database is hosted on the local server, allowing you to directly view, edit, and query the tables used in your system.

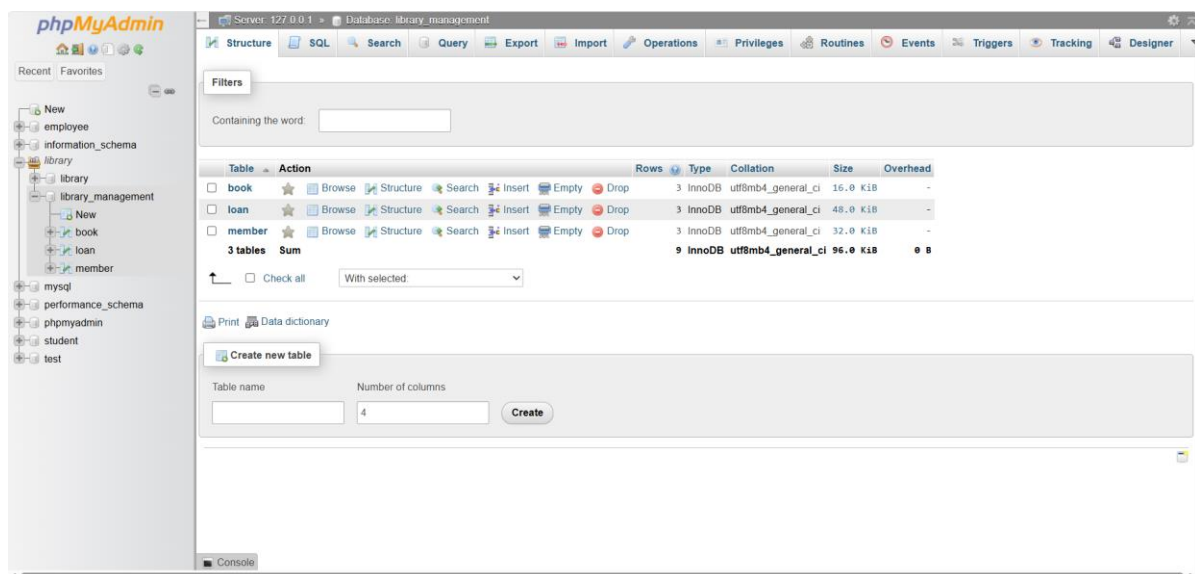


Figure 5.4: Structure tab

This image displays the **Structure tab** of your `library_management` database in **phpMyAdmin**, explicitly showing the three core tables you created—**book**, **loan**, and **member**—along with their record counts and storage details.

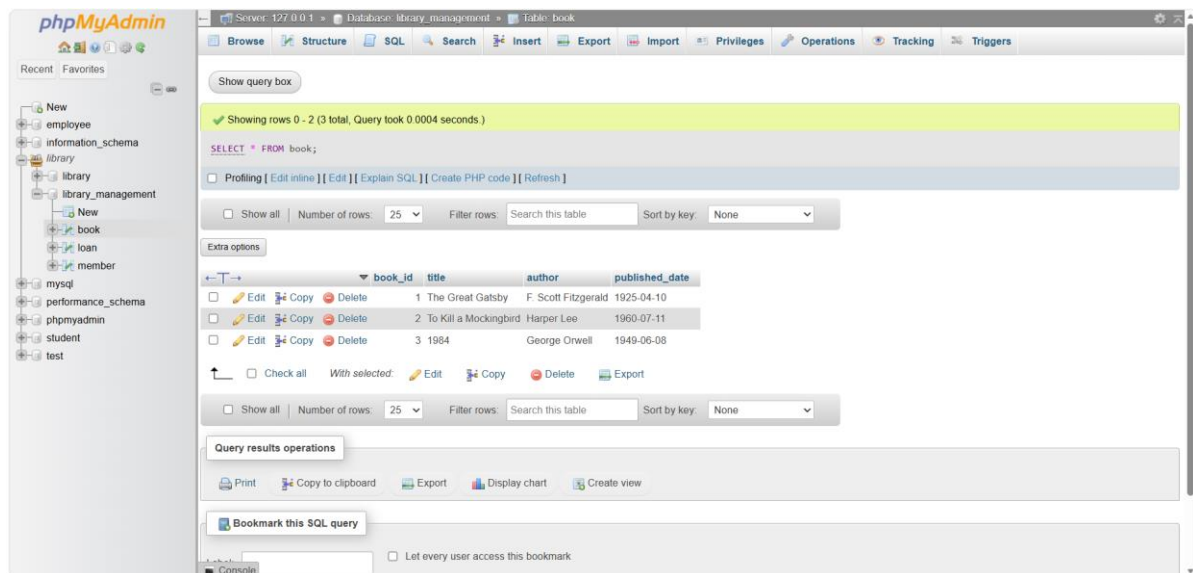


Figure 5.5: **Browse tab**

This image shows the **Browse tab** for the **book** table in phpMyAdmin, displaying the successfully inserted records for your library's inventory, including titles like *The Great Gatsby* and *1984* along with their respective IDs, authors, and publication dates.

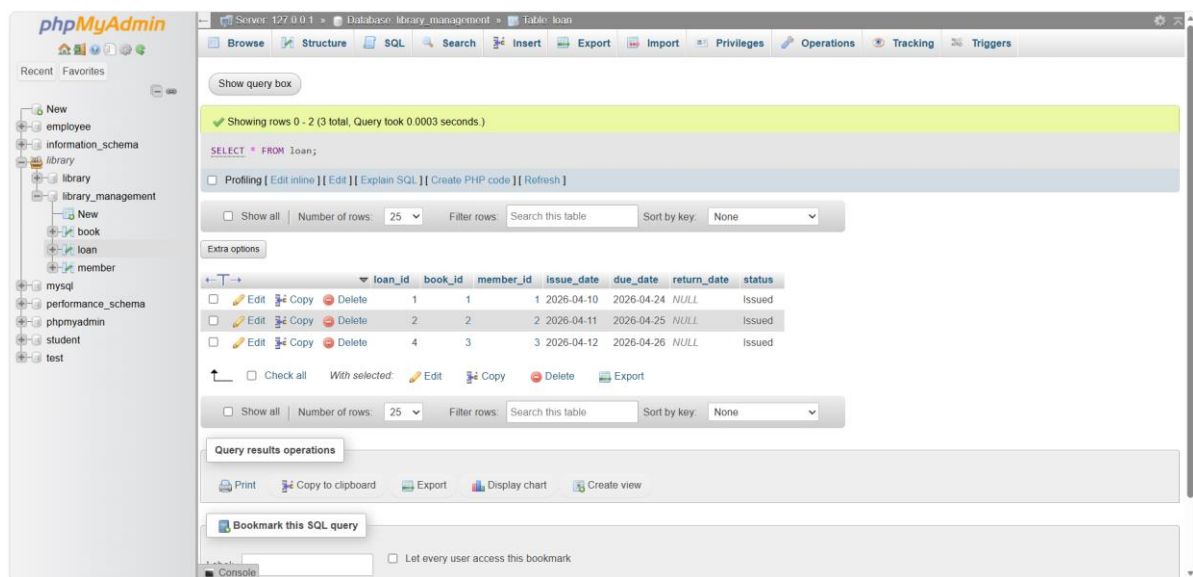


Figure 5.6: **Browse tab**

This image displays the **Browse tab** for the **loan** table in phpMyAdmin, showing the active circulation records where specific books (via `book_id`) are linked to specific members (via `member_id`) with their corresponding issue dates, due dates, and current "Issued" status.

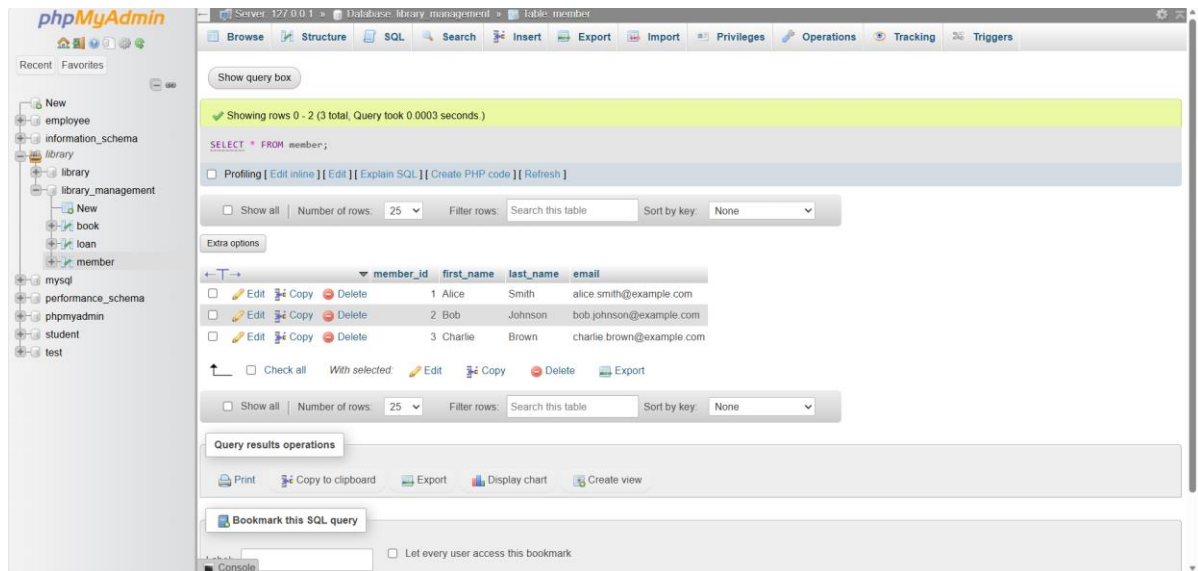


Figure 5.7: Browse Tab

This image shows the **Browse** tab for the member table in phpMyAdmin, displaying the registered users of your library system (Alice, Bob, and Charlie) along with their unique member IDs and email addresses, confirming that your member data is correctly stored and ready for borrowing.

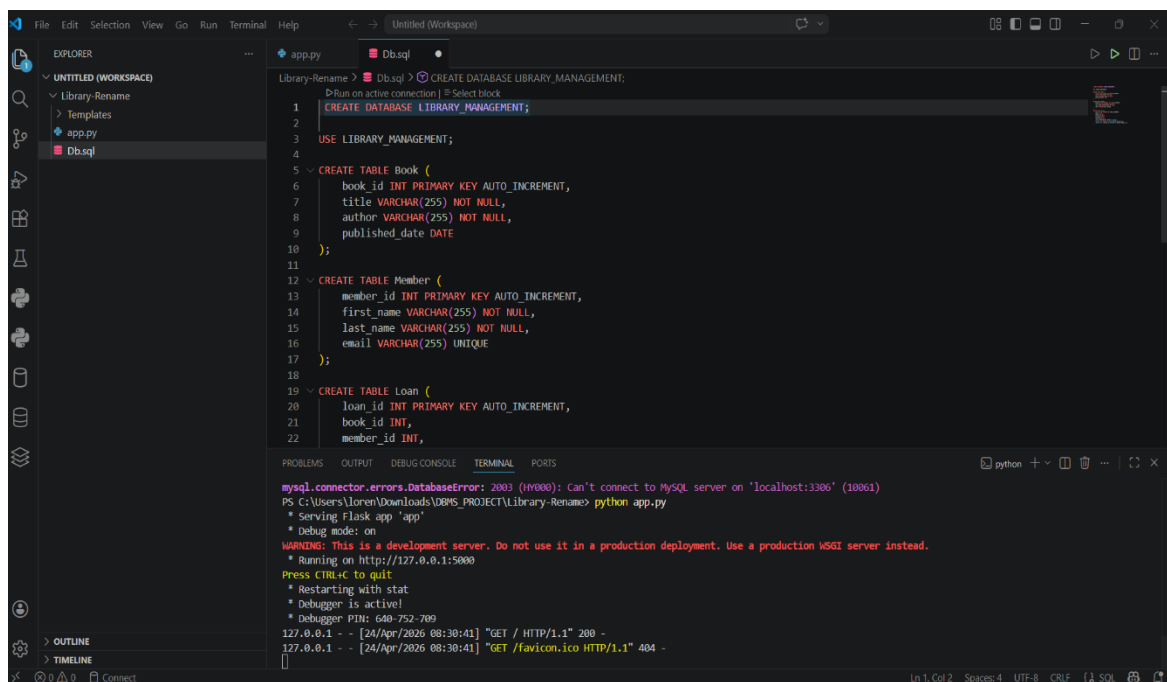


Figure 5.8: Front-End Development Interface

The above image shows the **front-end interface development of the Database Management System mini project using Python and Flask in Visual Studio Code**, where the application is executed through app.py and connected with the SQL database file Db.sql, with the web server successfully running on localhost.

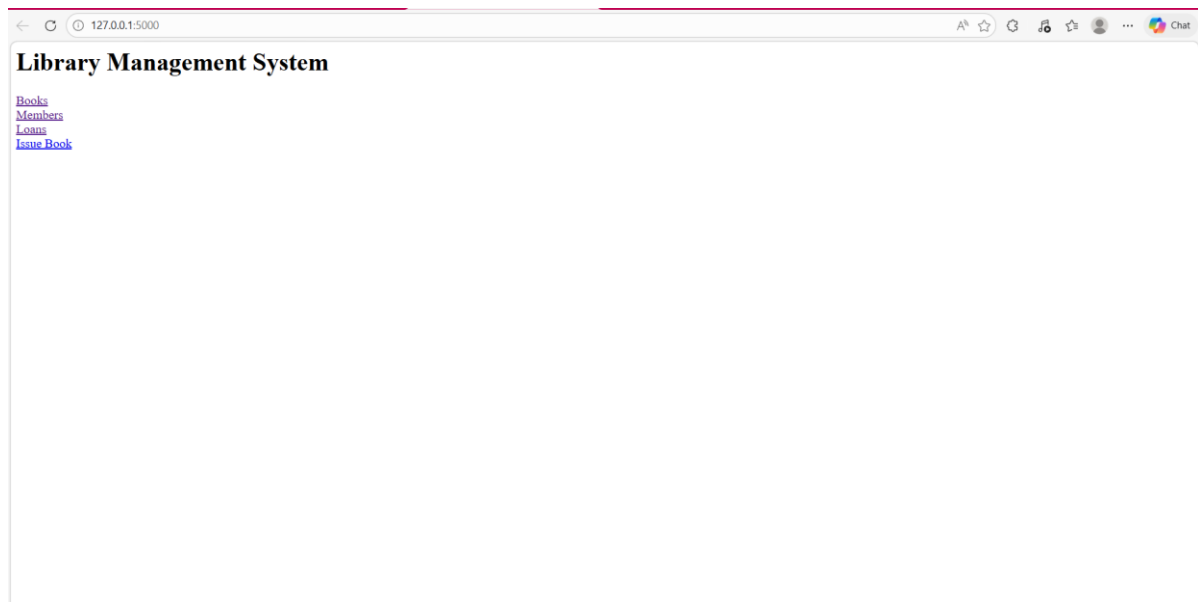


Figure 5.8: Home Page of the Library Management System web application developed using Python and Flask

The image shows the **home page of the Library Management System web application developed using Python and Flask**, running on the local server (127.0.0.1:5000). The interface contains the main title **“Library Management System”** along with navigation links such as **Books, Members, Loans, and Issue Book**, which allow the user to access different modules of the system for managing book records, member details, loan transactions, and book issuance operations.

Chapter 6

Conclusion and Future work

The development of the **Library Management System** successfully demonstrates the integration of a relational database with a functional management interface. By utilizing the **XAMPP** stack and **MySQL**, the project provides a reliable solution for automating core library tasks such as inventory tracking, member registration, and loan processing. The transition from a manual system to this digital schema ensures **data consistency**, eliminates redundant records, and provides real-time visibility into book availability.

To further enhance the capabilities of the system, the following developments are proposed:

- **Fine & Penalty Module:** Implement an automated calculation system to track overdue books and generate late fee invoices for members based on the `due_date` and `return_date`.
- **Email Notifications:** Integrate an automated mailing service to send reminders to members three days before their due date and immediate alerts for overdue items.
- **Digital Reservations:** Add a "Reservation" feature allowing members to book a currently issued book in advance, placing them in a queue for the next available copy.
- **Advanced Analytics Dashboard:** Develop a graphical reporting module for librarians to visualize monthly borrowing trends, popular genres, and peak activity times using chart libraries.
- **Barcode/QR Integration:** Incorporate barcode scanning functionality to allow staff to issue or return books instantly by scanning physical codes, further reducing manual entry errors.

References

[1] Database Systems: Models, Languages, Design and Application Programming, Ramez Elmasri and Shamkant B. Navathe, 7th Edition, Pearson.

[2] Database Management Systems, Raghu Ramakrishnan and Johannes Gehrke, 3rd Edition, 2014, McGraw Hill.